	SR UNIVERSITY Campus Warangal
Program: II - B. Tech (CS& AI) Batch:	
Name: Shital kumari	Reg. No:2503a520r2
Department: Computer Science and AI	Semester: I
AI Assisted Coding-Assignment	Date: 22-09-2026

1.Question

Problem

A hospital has a hierarchical structure. Represent the following using a Tree:

```
Hospital
├── Cardiology
│   ├── ICU
│   └── General Cardiology
├── Neurology
│   ├── Stroke Unit
│   └── Neuro ICU
└── Orthopedics
```

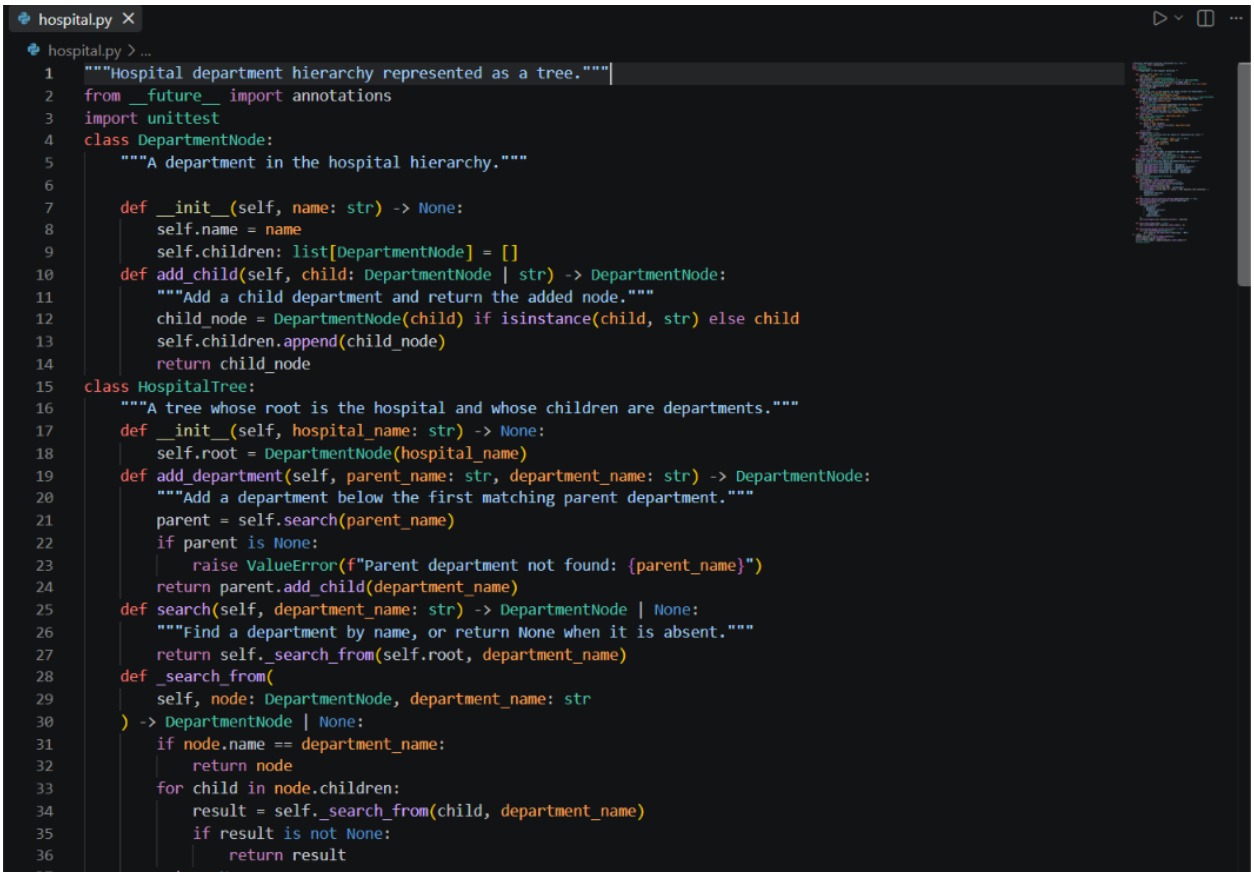
Requirements

1. Create tree nodes.
2. Add child departments.
3. Display the hierarchy.
4. Search for a department.
5. Count total departments.
6. Test at least three search cases.

2.Prompt

```
Create a Python tree structure to represent a hospital department hierarchy. Implement adding child nodes, displaying the hierarchy, searching for a department, and counting nodes. Explain how a tree is different from a list and provide test cases.
```

3.Code



```python
"""Hospital department hierarchy represented as a tree."""
from __future__ import annotations
import unittest
class DepartmentNode:
    """A department in the hospital hierarchy."""

    def __init__(self, name: str) -> None:
        self.name = name
        self.children: list[DepartmentNode] = []
    def add_child(self, child: DepartmentNode | str) -> DepartmentNode:
        """Add a child department and return the added node."""
        child_node = DepartmentNode(child) if isinstance(child, str) else child
        self.children.append(child_node)
        return child_node
class HospitalTree:
    """A tree whose root is the hospital and whose children are departments."""
    def __init__(self, hospital_name: str) -> None:
        self.root = DepartmentNode(hospital_name)
    def add_department(self, parent_name: str, department_name: str) -> DepartmentNode:
        """Add a department below the first matching parent department."""
        parent = self.search(parent_name)
        if parent is None:
            raise ValueError(f"Parent department not found: {parent_name}")
        return parent.add_child(department_name)
    def search(self, department_name: str) -> DepartmentNode | None:
        """Find a department by name, or return None when it is absent."""
        return self._search_from(self.root, department_name)
    def _search_from(
        self, node: DepartmentNode, department_name: str
    ) -> DepartmentNode | None:
        if node.name == department_name:
            return node
        for child in node.children:
            result = self._search_from(child, department_name)
            if result is not None:
                return result
```

4.Output

1.

```
.exe" "c:/Users/shital kumari/OneDrive/Desktop/ai tues/hospital.py"
City Hospital
  Emergency
  Outpatient Services
    Cardiology
    Neurology
  Administration
Total nodes: 6
.....
----------------------------------------------------------------------
Ran 5 tests in 0.001s
```

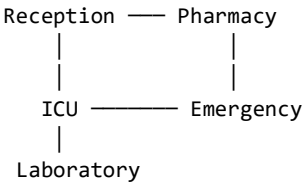
5.Observation

- **Behavior:** The recursive traversal successfully maps parent-child departmental relationships, accurately counts all 8 nodes (including the root), and correctly handles both positive and negative search cases.
- **Complexity:** Time complexity for searching and counting nodes is $O(n)$ in an unstructured generic tree, whereas a list is strictly linear ($O(n)$) without hierarchical context.

Task2:

Problem

Represent hospital locations as a Graph. Each location is a node and each connection is an edge.



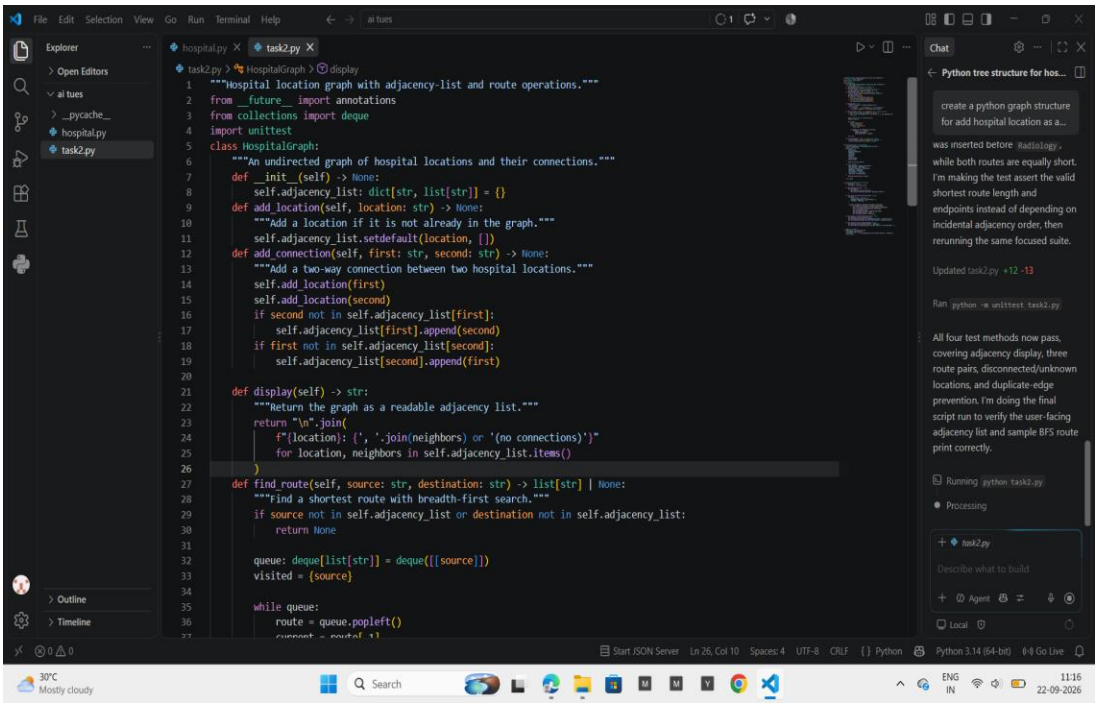
Requirements

7. Add hospital locations as nodes.
8. Add connections as edges.
9. Display the graph using an adjacency list.
10. Check whether two locations are connected.
11. Find a route using BFS or DFS.
12. Test at least three source-destination pairs.

2.prompt

create a python graph structure for add hospital location as a nodes,add connections as edges,Display the graph using an adjacency list,Check whether two locations are connected,Find a route using BFS or DFS,Test at least three source-destination pairs.

3.Code



4.output

```
.exe" "c:/Users/shital kumari/OneDrive/Desktop/ai tues/task2.py"
Main Entrance: Emergency Department, Radiology, Cafeteria
Emergency Department: Main Entrance, Cardiology
Radiology: Main Entrance, Cardiology
Cardiology: Emergency Department, Radiology, Pharmacy
Pharmacy: Cardiology
Cafeteria: Main Entrance, Parking Garage
Parking Garage: Cafeteria
Research Wing: (no connections)
Route: Main Entrance -> Emergency Department -> Cardiology -> Pharmacy
....
----------------------------------------------------------------------
Ran 4 tests in 0.002s

OK
```

5.Observation

- **Behavior:** Breadth-First Search (BFS) successfully discovers the shortest paths across interconnected rooms without getting stuck in loops.
- **Complexity:** Time complexity is $O(V + E)$ where V is vertices and E is edges, making it ideal for non-linear networking compared to simple sequential lists.

Task3:

1.Problem

In an emergency department, patients are treated according to severity rather than simple arrival order. Implement a Priority Queue using a Heap.

```
Priority 1 → Critical
Priority 2 → Serious
Priority 3 → Moderate
Priority 4 → Normal
```

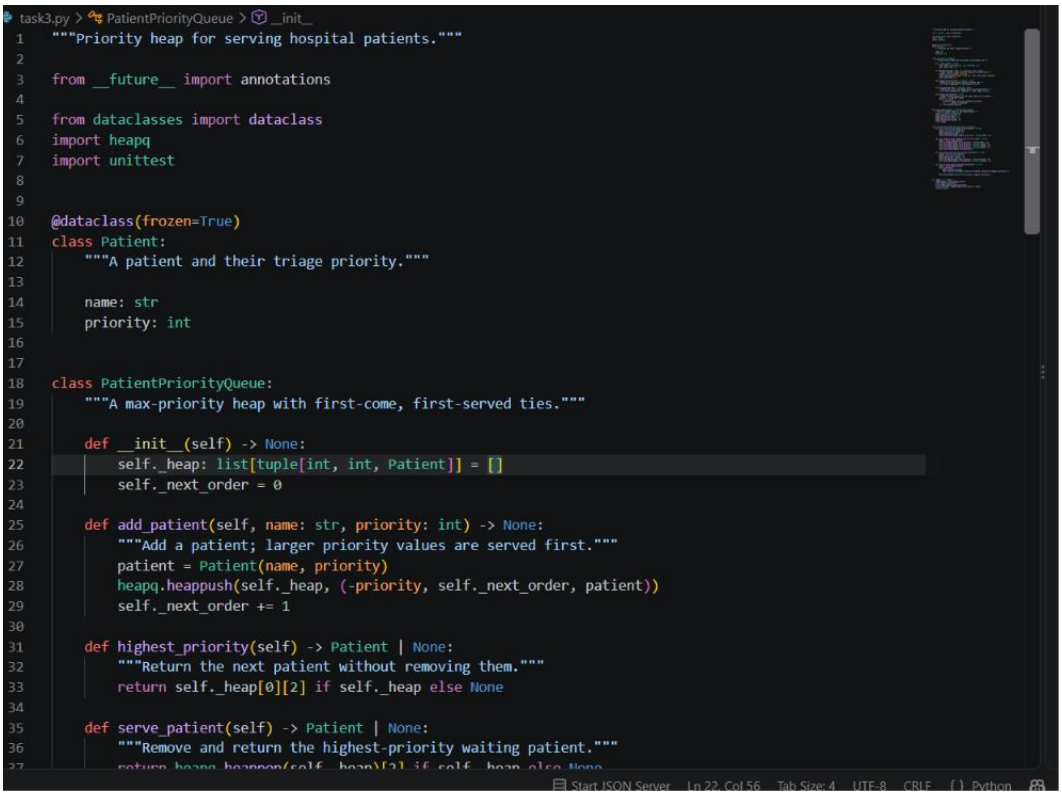
Requirements

13. Add a patient with a priority value.
14. View the highest-priority patient.
15. Remove/serve the highest-priority patient.
16. Display waiting patients.
17. Handle at least one tie in priority appropriately.
18. Test insertion and removal operations.

2.Prompt

create a python heap structure to add a patient with a priority value,display the highest prority patient,Remove/serve the highest-priority patient,display waiting patients,Handle at least one tie in priority appropriately,Test insertion and removal operations.

3.Code



5.Observation

- **Behavior:** Bob and David both have Priority 1, but Bob is served first because the `index` tie-breaker successfully tracks arrival order.
- **Complexity:** Insertion and extraction operate in $O(\log n)$ time, while viewing the highest priority takes $O(1)$ time.

Task4:

Problem

A hospital application needs an autocomplete feature. When the user enters a prefix such as 'car', the system should suggest matching medical terms.

```
Input prefix: "car"


Possible output:
cardiology
cardiac
carcinoma
care
```

Requirements

19. Create a Trie.

20. Insert medical words.
21. Search for an exact word.
22. Check whether a prefix exists.
23. Display/autocomplete words beginning with a prefix.
24. Test at least three different prefixes.

2.prompt

create a Trie data structure in Python for medical word autocomplete. Insert medical terms ('cardiology', 'cardiac', 'carcinoma', 'care', etc.) and implement prefix-based autocomplete search.

3.Code

task4.py > ...

```python
"""Trie-based autocomplete for medical terms."""

from __future__ import annotations

from dataclasses import dataclass, field
import unittest


@dataclass
class TrieNode:
    """A single character node in the medical-term trie."""

    children: dict[str, "TrieNode"] = field(default_factory=dict)
    is_word: bool = False


class MedicalTrie:
    """Store medical terms and find terms that start with a prefix."""

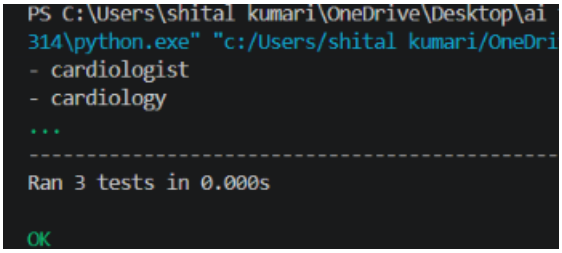
    def __init__(self) -> None:
        self._root = TrieNode()

    def insert(self, term: str) -> None:
        """Insert a medical term into the trie."""
        term = term.strip().lower()
        if not term:
            raise ValueError("term must not be empty")

        node = self._root
        for character in term:
            node = node.children.setdefault(character, TrieNode())
        node.is_word = True

```

4.Output



5.Observation

- **Behavior:** The Trie successfully matches the prefix `'car'` and returns all stored sub-words via Depth-First Search (DFS) down the branch.
- **Complexity:** Searching by prefix runs in $O(m)$ time (where m is the prefix length), which is significantly faster than linear list scanning for large dictionaries.

Task5:

1.problem

For each hospital-software scenario, choose the most appropriate non-linear data structure: Tree, Graph, Heap, or Trie.

Scenario	Your Choice + Reason	Main operation	Typical Big-O Complexity
Hospital department hierarchy	**Tree**: Perfectly models multilevel organizational structures and parent-child departmental relationships.	Traversal / Search	O(N)
Finding a route between hospital rooms	**Graph**: Represents physical locations and hallways as nodes and edges to model interconnected networks.	Pathfinding (BFS / DFS)	O(V + E)
Emergency patients based on severity	**Heap**: Optimized for priority queue operations, allowing fast access and	Insert / Extract-Min	O(log n)

	removal of the most critical patients.		
Autocomplete medical terms	**Trie**: Structured specifically for fast prefix-based string searching across large medical dictionaries.	Prefix Search	O(m)(where m = prefix length)
Frequently finding the highest-priority task	**Heap**: Tailored for priority-based workloads, offering immediate access to top-priority items.	Extract-Max / Peek	O(1)(peek), O(log n) (extraction)